

# PENETRATION TEST REPORT

## Server-Side Request Forgery (SSRF) Assessment

|                      |                                                                        |
|----------------------|------------------------------------------------------------------------|
| Target Application   | DVWA (Damn Vulnerable Web Application) v1.10 Development               |
| Target IP Address    | 10.112.155.32                                                          |
| Vulnerability Tested | Server-Side Request Forgery (SSRF) via File Inclusion 'page' parameter |
| Assessment Date      | 25 June 2026                                                           |
| Assessed By          | Yousef Gamal Sleem                                                     |
| Tools Used           | Burp Suite Community Edition v2026.1.5, Firefox                        |
| Classification       | Confidential — Educational Lab Report                                  |

# 1. Executive Summary

A Server-Side Request Forgery (SSRF) assessment was conducted against the DVWA File Inclusion module at 10.112.155.32, which was used as the SSRF-capable vector since DVWA does not ship a dedicated SSRF module. The page parameter of the File Inclusion module is passed into a server-side include() operation, meaning the DVWA server itself — not the tester’s browser — resolves and fetches the supplied location, which is the defining characteristic of SSRF (CWE-918).

Testing was performed against the Impossible security level. Two exploitation attempts were made using Burp Suite Repeater: a local path-traversal payload targeting /etc/passwd, and a remote-URL payload targeting a loopback address (http://127.0.0.1/). Both attempts were unsuccessful — the server did not process the injected page value and instead returned the content of the legitimate, default file (file1.php), confirming that the Impossible level’s input validation correctly rejected both payloads.

Directory enumeration of the target also confirmed the underlying server stack: Apache 2.4.7 on Ubuntu, consistent with the DVWA lab environment.

## 1.1 Summary of Findings

|                       |                                                   |
|-----------------------|---------------------------------------------------|
| Target                | 10.112.155.32 — DVWA                              |
| Module                | File Inclusion (used as SSRF vector)              |
| Vulnerability Tested  | Server-Side Request Forgery in the page parameter |
| Security Level Tested | Impossible                                        |
| Result                | No bypass achieved — both payloads were rejected  |
| Server Stack          | Apache/2.4.7 (Ubuntu), PHP/5.5.9-1ubuntu4.26      |
| Assessment Date       | 25 June 2026                                      |

## 2. Scope & Methodology

### 2.1 Scope

|                            |                                          |
|----------------------------|------------------------------------------|
| <b>Application</b>         | DVWA v1.10 Development                   |
| <b>Platform</b>            | Isolated lab environment                 |
| <b>Target IP</b>           | 10.112.155.32                            |
| <b>Target URL</b>          | http://10.112.155.32/vulnerabilities/fi/ |
| <b>Server OS</b>           | Linux Ubuntu                             |
| <b>Web Server</b>          | Apache 2.4.7                             |
| <b>PHP Version</b>         | 5.5.9-1ubuntu4.26                        |
| <b>Vulnerability Class</b> | Server-Side Request Forgery (CWE-918)    |
| <b>Level Tested</b>        | Impossible                               |

### 2.2 Methodology

The assessment followed a structured exploitation methodology using Burp Suite Repeater:

1. Reconnaissance — Enumerate the target web root to confirm the server stack and DVWA installation path.
2. Baseline Request — Capture a legitimate request to the File Inclusion module (?page=file1.php) in Burp Suite Repeater.
3. Local Path Traversal Attempt — Modify the request to target /etc/passwd via a traversal payload and resend at the Impossible security level.
4. Remote/Internal URL Inclusion Attempt — Modify the request to target a loopback address (127.0.0.1) to test whether the server would issue an internal-facing request, and resend at the Impossible security level.
5. Response Analysis — Compare each response against the baseline to determine whether the injected value was processed or rejected.

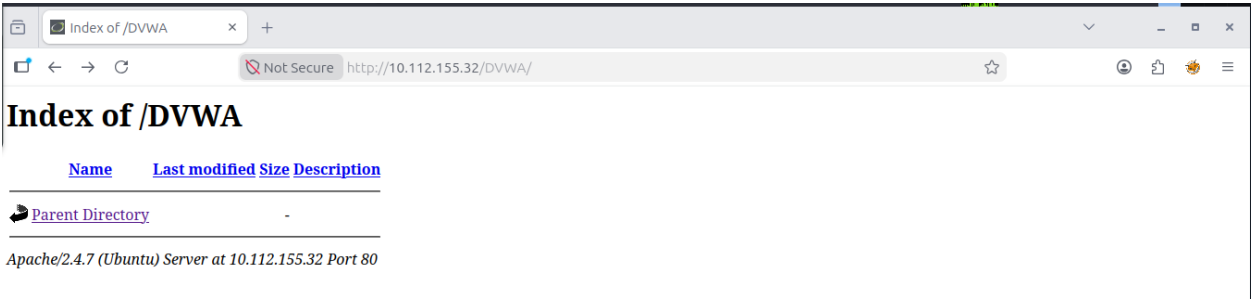
### 3. Findings

#### 3.1 Reconnaissance — Server Stack Confirmation

|               |                                                                                   |
|---------------|-----------------------------------------------------------------------------------|
| URL           | http://10.112.155.32/DVWA/                                                        |
| Server Banner | Apache/2.4.7 (Ubuntu) Server                                                      |
| Observation   | Directory listing enabled; confirms DVWA install path and underlying server stack |

Browsing to the DVWA web root returned a directory index confirming the target server is running Apache/2.4.7 on Ubuntu, consistent across all subsequent requests (also visible in the X-Powered-By and Server response headers captured in Burp Suite).

#### Evidence — Figure 1: Apache Directory Listing

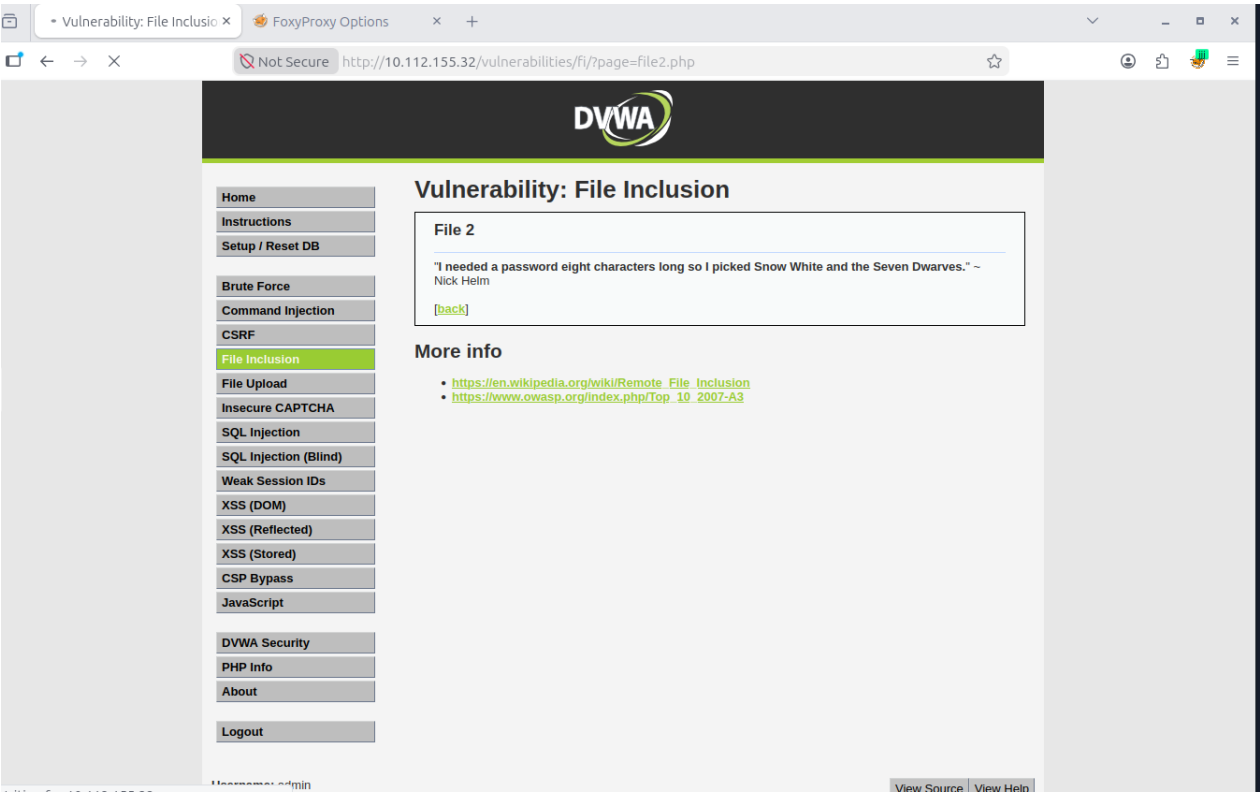


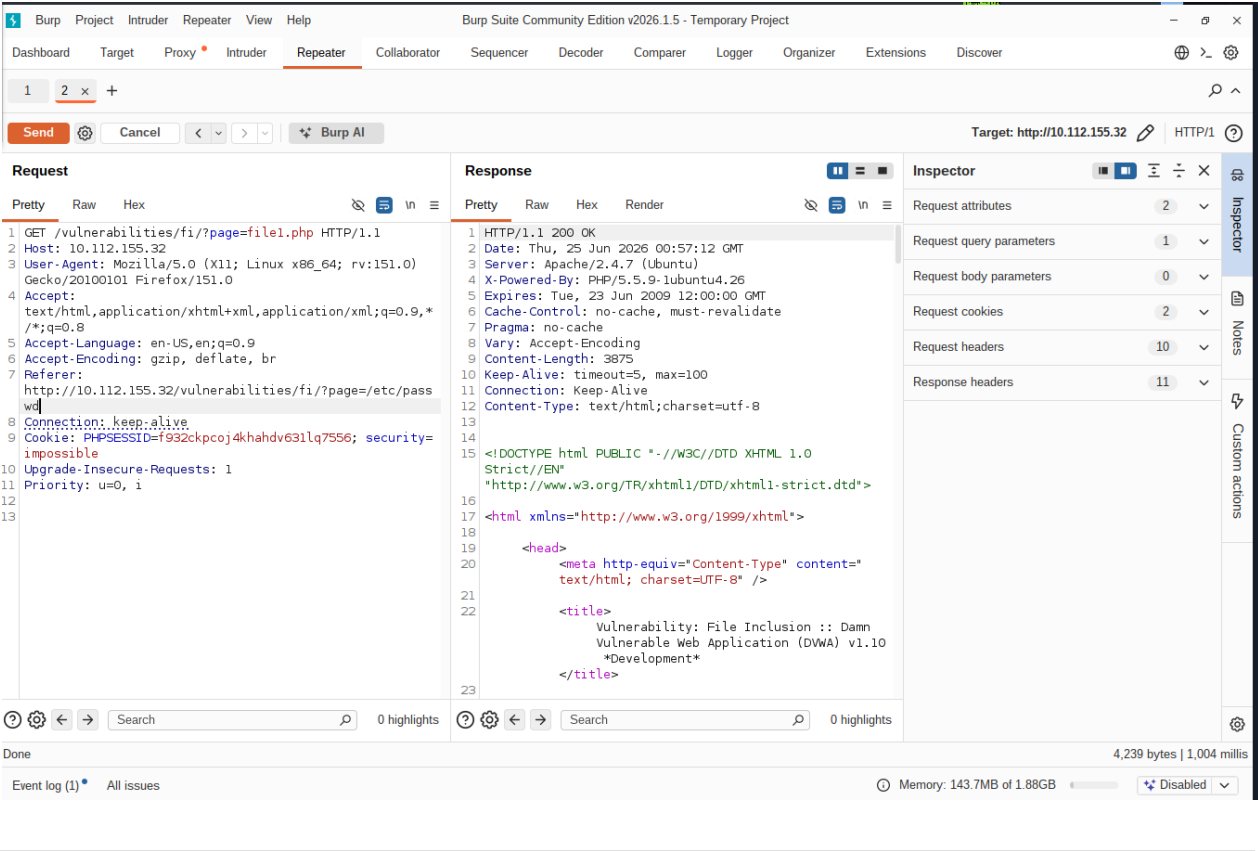
#### 3.2 SSRF Attempt #1 — Local Path Traversal (Impossible Level)

|                |                                                                              |
|----------------|------------------------------------------------------------------------------|
| Severity       | Informational (rejected)                                                     |
| Parameter      | page (GET parameter, via Referer header in this capture)                     |
| Endpoint       | /vulnerabilities/fi/?page=file1.php                                          |
| Security Level | Impossible                                                                   |
| Payload Target | /etc/passwd via path traversal                                               |
| Result         | Rejected — server returned the legitimate file1.php content, not /etc/passwd |

The request was sent to the File Inclusion endpoint with the page parameter referencing the traversal target (?page=/etc/passwd). At the Impossible security level, DVWA validates the page parameter against a fixed allow-list of permitted filenames (include.php, file1.php, file2.php, file3.php) and silently falls back to a safe default when the supplied value does not match. The response confirmed this: HTTP/1.1 200 OK was returned with the standard File Inclusion page content, with no trace of /etc/passwd in the response body.

Evidence — Figure 2: Burp Suite Repeater — Local Path Traversal Attempt (Impossible)





### 3.3 SSRF Attempt #2 — Internal/Loopback URL Inclusion (Impossible Level)

|                |                                                                                                                    |
|----------------|--------------------------------------------------------------------------------------------------------------------|
| Severity       | Informational (rejected)                                                                                           |
| Parameter      | page (GET parameter, via Referer header in this capture)                                                           |
| Endpoint       | /vulnerabilities/fi/?page=file1.php                                                                                |
| Security Level | Impossible                                                                                                         |
| Payload Target | http://127.0.0.1/ (loopback / internal address)                                                                    |
| Result         | Rejected — server returned the legitimate file1.php content; no server-side fetch of the internal address occurred |

A second attempt targeted a loopback address (`http://127.0.0.1/`) to test whether the server could be coerced into making an internal-facing request — the classic SSRF impact scenario. As with the traversal attempt, the Impossible level’s strict allow-list rejected the supplied value outright and returned the default page content. No evidence of an outbound or internal request being issued by the server was observed in the response.

#### Request

```
GET /vulnerabilities/fi/?page=file1.php HTTP/1.1
```

```
Host: 10.112.155.32
Referer: http://10.112.155.32/vulnerabilities/fi/?page=http://127.0.0.1/
Cookie: PHPSESSID=f932ckpc0j4khahdv631lq7556; security=impossible
```

## Evidence — Figure 3: Burp Suite Repeater — Internal/Loopback URL Attempt (Impossible)

The screenshot shows the Burp Suite Repeater interface. The request is a GET request to `http://10.112.155.32/vulnerabilities/fi/?page=file1.php`. The response is a 200 OK status with a content type of `text/html; charset=utf-8`. The response body contains the following HTML:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  </head>
  <title>
    Vulnerability: File Inclusion :: Damn Vulnerable Web Application (DVWA) v1.10
    *Development*
  </title>
```

The status bar at the bottom indicates that the request was successful (200 OK) and that the response was 4,239 bytes in size.

## 3.4 Why Both Attempts Were Rejected

At the Impossible security level, DVWA validates the page parameter using a strict, fixed allow-list (a PHP `in_array()` check against the literal values `include.php`, `file1.php`, `file2.php`, and `file3.php`). Any value outside this list — including both payloads used in this assessment — is rejected before reaching the `include()` call, and the application silently substitutes the default page instead of raising an error. This explains why both requests returned HTTP/1.1 200 OK with ordinary File Inclusion content rather than an error message, a file dump, or a sign of an outbound connection.

- This allow-list approach is the recommended pattern for preventing SSRF/LFI: restrict the parameter to a closed set of known-safe values, rather than attempting to blocklist specific traversal sequences or protocol prefixes (the approach used at DVWA's Low and Medium levels, which are bypassable).

## 4. Recommendations

### 4.1 Maintain Strict Allow-Listing for Path/URL Parameters

The Impossible level's approach — validating the page parameter against a fixed, explicit array of permitted values — is the correct general pattern and should be the baseline for any feature that resolves a user-influenced path or URL on the server side. This control should remain in place; the Low and Medium implementations (no validation, and naive blocklist filtering, respectively) should not be used in production code.

### 4.2 Disable Remote File Inclusion at the PHP Configuration Level

Set `allow_url_include = Off` in `php.ini` so that `include()/require()` cannot fetch remote URLs at all, removing the underlying SSRF-via-include() vector regardless of any application-level filtering.

### 4.3 Restrict Outbound Server-Side Requests at the Network Layer

Apply egress filtering so the application server cannot reach internal-only or loopback addresses, providing a safety net even if an application-level SSRF vulnerability is later discovered in a different feature.

### 4.4 Continue Testing Lower Security Levels

This assessment only exercised the Impossible level. To fully characterize the risk this vulnerability class poses to the application, the Low, Medium, and High security levels should also be tested, since DVWA's Low level is expected to have no validation at all and the Medium level's blocklist-style filtering is typically bypassable using path-traversal sequence manipulation.



## 5. Conclusion

This assessment tested SSRF-style exploitation against the DVWA File Inclusion module at the Impossible security level using both a local path-traversal payload and an internal/loopback URL payload. Neither attempt succeeded: DVWA's strict, fixed allow-list correctly rejected both payloads and returned the default, legitimate page content instead. This confirms that the Impossible level's input-validation approach is effective against the SSRF vectors tested in this assessment.

Because testing was limited to the Impossible level, this report should be treated as a partial assessment. The Low, Medium, and High security levels remain untested and are very likely to be exploitable, based on DVWA's documented design and the equivalent findings from related modules. Testing those levels is recommended as a follow-up to provide a complete risk picture for this vulnerability class.